

Optimization of Multiplexer Combination in RTL Logic Synthesis

Zicheng Wang
School of Microelectronics
Xidian University
Xian, China
wangzchl@gwxeda.com

Hailong You*
School of Microelectronics
Xidian University
Xian, China
hlyou@mail.xidian.edu.cn

Jie Wang
School of Microelectronics
Xidian University
Xian, China
wangjie@gwxeda.com

Meihua Liu
Institute of Advanced Computing and
Digital Engineering
Shenzhen Institute of Advanced
Technology Chinese Academy of
Sciences
Shenzhen, China
liumh@gwxeda.com

Yu Su
Shenzhen GWX Technology Co.,Ltd.
Shenzhen, China
suyu@gwxeda.com

Yong Zhang
Institute of Advanced Computing and
Digital Engineering
Shenzhen Institute of Advanced
Technology Chinese Academy of
Sciences
Shenzhen, China
zhangyong@siat.ac.cn

Abstract—The paper presents an optimizable multiplexer combination in logic synthesis of RTL level circuits. There are often some redundant multiplexers in the circuit, the deletion or optimization of these multiplexers does not affect the design behavior of circuits. Therefore, we propose a special multiplexer combination. It can be optimized on the premise that the logic function of the circuit remains unchanged, and it can also be merged with other traditional optimizable multiplexers to further optimize the circuit. In the paper, we use examples to optimize and verify, the results show that the optimizable combination can effectively reduce the number of multiplexers. The results show that the optimizable combination can significantly reduce the number of multiplexer elements, and appears more frequently and probably in the circuit. The optimizable combination is feasible and practical in the logic synthesis of circuits, which can effectively reduce the number of elements and design scale of the circuit.

Keywords—Multiplexer, Optimization, Synthesis, Connectivity Check

I. INTRODUCTION

Driven by Moore's Law [1], large-scale digital circuits are becoming ever more complex. By 2012, billions of levels of transistorized processors were commercially available. As an important element in digital circuit, Multiplexer(MUX) is a kind of standard complex gate, which is usually used in data path logic, and occupies a great proportion in the number of elements in digital circuit. MUX are used extensively in a number of applications including processors, processor busses, network switches, and even DSP designs with resource sharing. In the field of FPGA, the optimization of the multiplexer is also the top priority [2-5].

In hardware description Language (HDL), MUX is usually described using if-else and case. When nesting of such statements occurs, it is reflected in the circuit as the MUX tree [2]. In a MUX tree, a special structure consisting of multiple MUX connected together is called the MUX combination. Due to the high frequency of if-else and case statements in RTL-level code, a large number of MUXs usually exist in the circuit after logical synthesis. And because of all kinds of nesting and else if statements, the MUX tree in the circuit becomes very common. As the circuit design logic becomes more and more complex, there

are often redundant MUXs in these MUX trees. The deletion or optimization of these MUXs does not affect the circuit function.

For this case, Many logical synthesis tools optimize MUX. The main approach is to specify some specific combinations of multiplexers that are optimizable and then find them out of the MUX trees based on their characteristics and optimize them accordingly. However, we find that in the traditional optimizable combinations, a certain input is usually treated as a dead end or constant in the optimization process, and deletes or changes it to achieve the purpose of optimizing the MUX trees. As a result, each optimizable combinations are discrete and unable to cooperate with each other to achieve further optimization.

In this paper, we propose an optimizable multiplexer combination in logic synthesis of RTL level circuits. we can optimize this combination to achieve the purpose of reducing the number of multiplexers in the circuit under the premise of the same logic function of the circuit. In addition, compared with the traditional combinations, this combination does not change the input, but directly changes the relationship between ports to reduce the number of MUX. The advantage of this is that the combination can be further optimized with the traditional multiplexer combinations.

Finally, in order to prove the feasibility and practicability of the optimizable combination, we use an example to conduct a control test before and after optimization. At the same time, we used the Connectivity Check (CC) function of the formal verification tool VC formal [6] to check the modified data path, ensuring that the circuit data path before and after optimization is functionally consistent.

II. TRADITIONAL OPTIMIZABLE MULTIPLEXER COMBINATION

At present, the logical synthesis tools on the market have abundant optimization methods. For example, Design Compiler [7], the most popular synthesis tool, designs optimization methods for different design levels: Architectural-Level optimization has implementation selection, data-path optimization, sharing common subexpressions, resource sharing, reordering operators and so on. Logic-Level optimization includes structuring optimization and flattening optimization. Gate-Level optimization will use devices with better performance

according to the device library provided by fab manufacturers. In addition, Yosys, an open source software, has various optimizers: removing unused signals and cells, const folding, merging repeated input, simplifying registers for constant input, resource sharing, and optimization of MUX tree, [8] etc.

Among these main optimization methods is the optimization of multiplexer combinations. Take optimization of MUX tree of Yosys as an example, there are several traditional optimization combinations:

- In the MUX tree, an input of a MUX is connected to a control port of a MUX (including itself) in the direction of its output, as shown in Fig.1.
- In the MUX tree, the control port of a MUX is connected to the control port of a MUX in the direction of its output, as shown in Fig.2.
- and there are simple cases where MUX has an invalid output or a constant control.

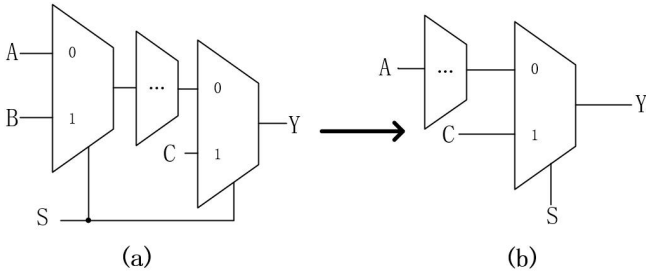


Fig.1. Traditional optimizable MUX combination 1, (a) optimizable combination, (b) optimized combination

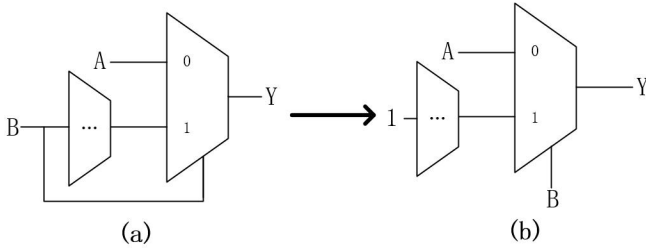


Fig.2. Traditional optimizable MUX combination 2, (a) optimizable combination, (b) optimized combination

We can find that the above commonly used optimizable combinations have modified the input of the MUX, but the logic of the MUX itself has not changed greatly. These combinations are often incompatible; they can be optimized simultaneously, but not further.

In addition, some common optimizations are to reduce the multiplexer to other elements. For example: MUX has a constant input, in which case MUX can be reduced to an OR gate or an AND gate, as shown in Fig.3.

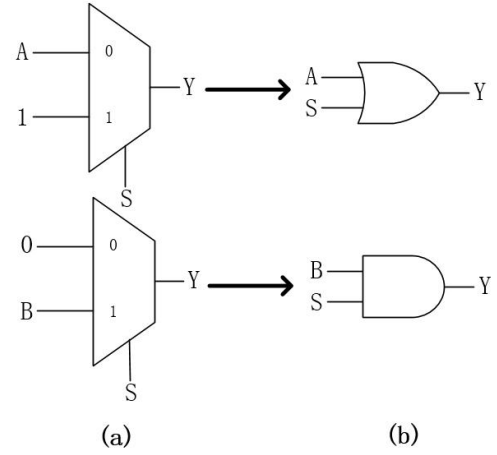


Fig.3. Traditional optimizable MUX combination 3, (a) optimizable combination, (b) optimized combination

III. PROPOSED OPTIMIZABLE MULTIPLEXER COMBINATION

According to the rules of the optimizable combinations described in Section II and the accumulation of practical circuits, we propose a new optimizable multiplexer combination. The main characteristic of this combination is that in the MUX tree, all MUX of a MUX's input port have exactly the same input, but their control signals are different, as shown in Fig.4.

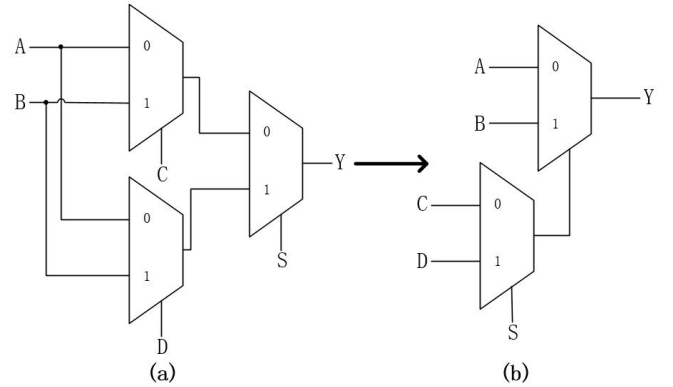


Fig.4. Proposed optimizable MUX combination, (a) optimizable combination, (b) optimized combination

We can see that this combination directly changes the relationship of the various ports in the multiplexer compared to the traditional combinations. The benefit of this is that a new MUX tree is generated at the MUX's control port. Therefore, this structure can be combined with other optimization structures to further optimize the MUX tree. For example, as shown in Fig.5, if the combination of Fig.1 and Fig.2 does not exist in the MUX tree, but after the optimization of Fig.4, the structure of the MUX tree changes, the optimizable combination of Fig.1 and Fig.2 may be generated again. This is an advantage that no other combination has.

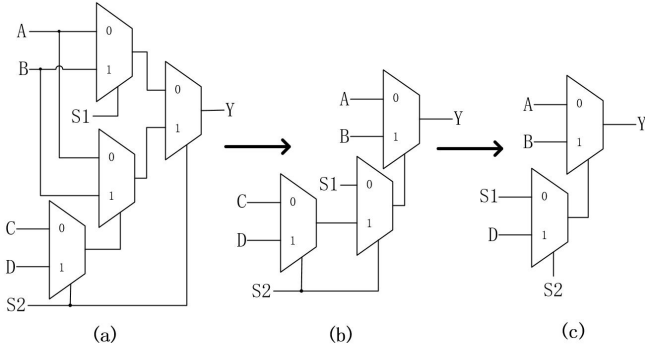


Fig.5. The MUX combination of Fig.2 and Fig.4. (a) optimizable combination, (b) proposed optimized combination, (c) traditional optimized combination

According to this feature, different combination variants will be generated when the combination is merged with some optimizable combinations. For example, when the combination is merged with the combination in Fig.3, the original optimizable combination will be changed into the combination structure as shown in Fig.6.

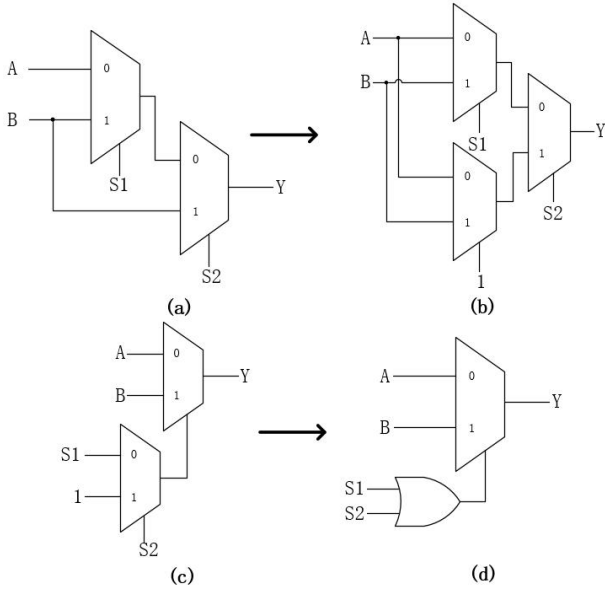


Fig.6. Variant of proposed optimizable MUX combination, (a) optimizable combination, (b) equivalent combination, (c) proposed optimized combination, (d) traditional optimized combination

The combination in Fig.6 (a) is not proposed above, but it can be expanded into the combination in (b). After the combination optimization in Fig.4, (c) is obtained, and the combination in Fig.3 appears, thus optimizing into (d). This optimizable combination from (a) to (d) forms variations based on Fig.3 and Fig.4. The difference between this combination and the previous ones is that it does not directly reduce the number of elements, but optimizes the MUX into a simpler OR gate. In addition, if the repeated input position is different, you can optimize by changing the control port to an AND gate or adding a NOT gate. For sure, The interior of the MUX is composed of multiple AND, OR and NOT gates, so decreasing a MUX but increasing an AND, OR, NOT gate can also decrease the number of gates in the circuit. More importantly, the combination is common in the circuit, so it is practical.

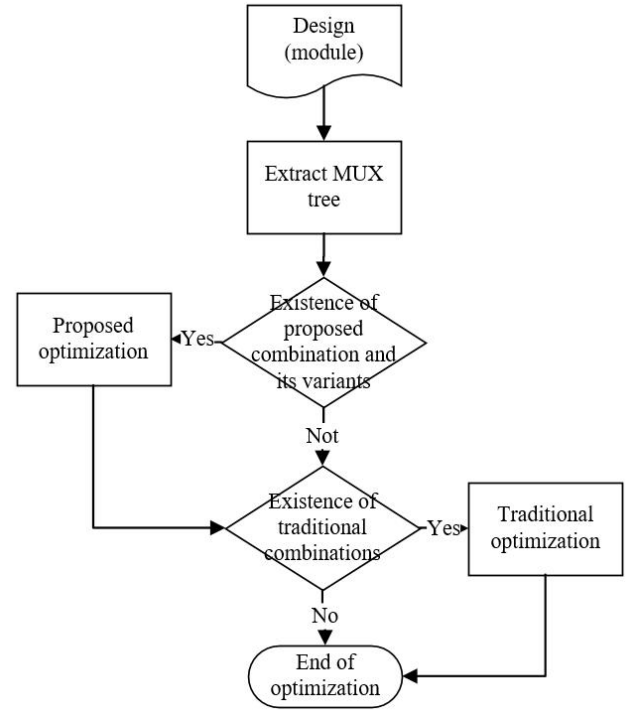


Fig.7. MUX tree optimization flowchart

According to the proposed combination and traditional combination mentioned above, we designed an optimization flow chart related to MUX tree, as shown in Fig.7. From the flow chart, it can be seen that the decision of the proposed combination must be before the traditional combinations. This is because the proposed combination may produce a traditional combination that is not present in the original circuit after optimization. In order to avoid repeated optimization and reach the optimal as soon as possible, we should first determine it in the MUX tree.

IV. EXPERIMENTAL RESULTS

For the optimizable multiplexer combination proposed in Fig.4 and Fig.6 in Section III, we use some examples from the open source test set on github [9]. The advantage of these test sets is that the circuit design has its own assertions, which can be used for subsequent property checks. To ensure consistency of circuit logic and function before and after optimization, we used a formal verification tool, VC formal, to verify properties and optimization-related paths of circuit, using the Formal Property Verification (FPV) function and Connectivity Check (CC) function, respectively. We have learned that in formal verification, FPV is mainly used to verify circuit functions and properties, which are generally presented in the form of assertions in HDL description [10]. And CC, as a function derived from FPV, mainly checks the path in the circuit to ensure that some ports and signals in the circuit exit paths that conform to circuit specifications. Therefore, when CC is used, the signals of source and destination can be specified in advance, so as to check the specified path [11]. In the paper, CC can be used to check the connectivity of the path where the optimizable combination is located. If the path connectivity is the same before and after the optimization, then the optimization has not changed the function of the circuit. In addition, we view the circuit through the combination of VC formal and Verdi, and use the relevant instructions to print the names and count quantities of the required elements.

We optimized and tested some examples, and the data results are shown in TABLE I. For each example, the first row is the number of circuit components before optimization, the second row is the number of components after optimization, and the third row is the change of the number of components before and after optimization, with positive numbers increasing and negative numbers decreasing.

TABLE I. GITHUB OPEN SOURCE TEST SET AVR

<i>Circuit</i> (test/opensource)	<i>Optimize</i>	<i>MUX</i>	<i>AND</i>	<i>OR</i>	<i>NOT</i>	<i>FPV</i> <i>/CC</i>
cache_coherenc e_three	before	347	48	48	52	✓
	after	303	58	50	60	
	change	↓44	↑10	↑2	↑8	
ethernet	before	226	14	51	26	✓
	after	186	38	61	28	
	change	↓40	↑24	↑10	↑2	
h_Arbiter	before	101	7	9	7	✓
	after	95	13	9	7	
	change	↓6	↑6	↑0	↑0	
h_FIFO	before	27	8	7	11	✓
	after	25	10	8	11	
	change	↓2	↑2	↑1	↑0	
h_Vsa16	before	48	10	21	18	✓
	after	47	11	21	18	
	change	↓1	↑1	↑0	↑0	
itc99_b12	before	295	13	36	45	✓
	after	279	14	36	47	
	change	↓16	↑1	↑0	↑2	
v_non- pipelined- microprocessor	before	56	5	5	5	✓
	after	48	13	5	5	
	change	↓8	↑8	↑0	↑0	
vcegar_arrays_i tc99_b12_p1	before	295	12	35	44	✓
	after	289	13	35	46	
	change	↓6	↑1	↑0	↑2	
vis_arrays_vsa1 6a_p1	before	70	15	12	10	✓
	after	65	16	12	11	
	change	↓5	↑1	↑0	↑1	
vis_arrays_vsaR _p01	before	73	16	10	13	✓
	after	70	17	10	13	
	change	↓3	↑1	↑0	↑0	

Through the analysis of Table I, we can see that in the circuit, we can reduce the logic complexity and the number of elements by reducing MUX but increasing the AND, OR and NOT gate. The results showed that after the FPV and CC of VC formal, the results are consistent before and after optimization, and the circuit logic did not change. In different circuits, the number of the optimizable combination is more or less, but all can achieve different degrees of

optimization. In addition, most of the optimizable combinations used in the above test examples are those mentioned in Fig.6, a variation that combines the proposed combination with the traditional combination. Therefore, in order to reflect the superiority of the optimizable combination proposed by us over the traditional combination, we also counted the number of traditional optimizable combinations, proposed optimizable combinations, and traditional optimizable combinations after proposed optimization in the examples, as shown in TABLE II.

TABLE II. THE NUMBER OF COMBINATIONS IN THE EXAMPLES

<i>Circuit</i> (test/opensource)	<i>Traditional</i>	<i>Proposed</i>	<i>Traditional</i> <i>after</i> <i>proposed</i>	<i>Traditional</i> <i>change</i>
cache_coherenc e_three	7	33	37	↑30
ethernet	20	54	72	↑52
h_Arbiter	18	6	21	↑3
h_FIFO	4	1	5	↑1
h_Vsa16	0	1	1	↑1
itc99_b12	63	23	71	↑8
v_non- pipelined- microprocessor	0	8	8	↑8
vcegar_arrays_i tc99_b12_p1	63	8	71	↑8
vis_arrays_vsa1 6a_p1	0	3	3	↑3
vis_arrays_vsaR _p01	0	1	1	↑1

The results show that in the above example, the proposed combination not only has a large number of occurrences, but also increases the number of traditional optimizable combinations after its optimization. After analysis, we find that this is because the optimizable combination described in Fig.6 is a common structure in circuits, which will not only apply the original structure proposed in Fig.4, but also increase the simple optimization in Fig.3. Similarly, our optimizable combination can be combined with other traditional combinations, which can also be further optimized, but only in limited quantities in the circuit.

In addition, in order to show the practicality of the combination, we also counted the number of circuits with proposed combination in all RTL designs of the open source test set. According to statistics, among 141 RTL design documents, 74 documents contain proposed optimizable combinations, accounting for about 52.5%. On analysis, we find that circuits that do not have this combination are usually small in size, probably because the length and branches of the MUX tree in small circuits are not large enough. In summary, it can be concluded that the proposed optimal combination can effectively reduce the number of MUX in the circuit.

V. CONCLUSION

In this paper, we propose an optimizable multiplexer combination in logic synthesis of RTL level circuits. The combination can be merged with traditional optimization combinations to further optimize the circuit, and different variations can be produced when merged with certain optimizable combinations. It can ensure that the number of

MUX and circuit size can be reduced without changing the circuit function in logic synthesis. In addition, the combination appears more frequently and probably in the circuit, so the optimization of it has feasibility and practicability.

ACKNOWLEDGMENT

Thanks to Prof. You Hailong and Dr. Liu Meihua for their guidance. Thanks to Prof. Zhang Yong. Thanks to Su Yu, Bai Geng, Huang Guoyong from Shenzhen GWX Technology Co.,Ltd., and Wang Jie from Xidian University, for their support.

REFERENCES

- [1] Moore G E. Cramming more components onto integrated circuits[J]. Proceedings of the IEEE, 1998, 86(1): 82-85.
- [2] Metzgen P, Nancekievill D. Multiplexer restructuring for FPGA implementation cost reduction[C]//Proceedings of the 42nd Annual Design Automation Conference. 2005: 421-426.
- [3] Pišteš P, Kolesár M, Jelemenská K. Optimization of multiplexer trees using modified truth table[C]//2010 International Conference on Applied Electronics. IEEE, 2010: 1-4.
- [4] Chen D, Cong J. Register binding and port assignment for multiplexer optimization[C]//ASP-DAC 2004: Asia and South Pacific Design Automation Conference 2004 (IEEE Cat. No. 04EX753). IEEE, 2004: 68-73.
- [5] Mitra S, Avya L J, McCluskey E J. Efficient multiplexer synthesis techniques[J]. IEEE Design & Test of Computers, 2000, 17(4): 90-97.
- [6] Duan L, Hu Y, Liu H, et al. An Efficient Formal Verification Method in I/O Multiplexing Module Based on VC Formal CC[C]//2020 IEEE 3rd International Conference on Electronics and Communication Engineering (ICECE). IEEE, 2020: 112-116.
- [7] NURPRASETYO E F, INOUE A, TOMIYAMA H, et al. Design Compiler Reference Manual Design Compiler Reference Manual[J]. IEICE transactions on electronics, 1998, 81(9): 1416-1423.
- [8] Wolf C. Yosys manual[J]. Retrieved January, 2021, 16: 2021.
- [9] Aman Goel. 2016. Abstractly Verifying Reachability. [https://github.com/aman-goel/avr. \(2023\).](https://github.com/aman-goel/avr. (2023).)
- [10] Fix L, McMillan K. Formal Property Verification[M]//EDA for IC System Design, Verification, and Testing. CRC Press, 2018: 20-1-20-14.
- [11] Yang P, Das P, Kao Y, et al. Formal Verification of Connections at SoC-level[J].